

미지의 경매장

AI 활용 기술

생성 모델은 표현하고, 엔진은 판정합니다 — 역할 분리, 2 단계 생성, 자동 검증과 폴백

1 AI 활용 목적

AI 는 플레이어가 추론할 재료를 런마다 새로 만들고, 게임의 판단은 사람이 설계한 엔진에 남겼습니다. 이 경계를 지키기 위해 생성 범위를 계약으로 고정하고, 결과를 자동으로 검증했습니다.

이렇게 나눈 이유는 다음과 같습니다. 미지의 경매장은 12 일 동안 하루 8 개, 총 96 개 경매 LOT 를 사용합니다. 각 LOT 에는 단순한 이름뿐 아니라 설명, 소문, 세트 힌트, NPC 반응과 시장 맥락이 필요합니다. 모든 문구를 고정 작성하면 반복 플레이어의 변화가 줄고, 무제한 생성에 맡기면 가격과 보상 같은 게임 규칙이 흔들릴 수 있습니다.

따라서 다음 네 가지를 목표로 AI 를 활용했습니다.

1. 같은 게임 규칙 안에서 런마다 다른 추론 재료를 제공합니다.
2. 반복 작성량을 줄이되, 게임의 핵심 판단은 사람이 설계한 엔진에 남깁니다.
3. AI 출력의 누락과 규칙 침범을 자동으로 검증합니다.
4. API 장애나 형식 실패가 실제 플레이를 막지 않도록 합니다.

AI 를 사용한 두 층

이 프로젝트에서 AI 는 성격이 다른 두 곳에 사용됩니다. 문서 대부분은 앞쪽을 설명하지만, 뒤쪽도 함께 밝힙니다.

런타임 생성		제작 단계 생성
언제	플레이할 때마다	개발 중 한 번
무엇	LOT 설명 · 소문 · 세트 힌트 · NPC 반응 · 사건 · 신문 리드	경매품 스프라이트 240 개, 씬 배경, 타이틀곡
검증	스키마 · ID · 금지 수치 · 문구 품질을 매 호출 자동 검사	사람이 확인 후 채택

런타임 생성		제작 단계 생성
실패 시	항목 단위 재생성 → 공급자 교체 → 폴백 문구	다시 생성해 교체

두 층 모두 결과물의 권리 조건을 확인하고 표기했습니다(9 장). 시각 에셋과 음악이 AI 생성이라는 사실도 감추지 않았습니다. 이 게임은 AI 로 만든 재료 위에서 AI 가 계속 이야기를 채우는 구조이며, 그 경계를 사람이 설계한 엔진이 지킨다는 것이 이 문서의 요지입니다.

2 설계 원칙 — 생성 모델은 표현하고, 엔진은 판정합니다

엔진이 소유하는 값	생성 AI 가 작성하는 내용
기준가 · 실제 가치	품목 표시 이름(<code>displayName</code>) · 설명(<code>description</code>)
등급 · 수요 계수	소문(<code>rumor</code>) · 세트 힌트(<code>setHint</code>)
보상 · 확률 · 배수	NPC 반응(<code>npcReaction</code>)
입찰 · 판매 · 대출 · 의뢰 규칙	런 전제(<code>premise</code>) · 12 일 시장 흐름(<code>marketArc</code>)
승리 · 실패 · 파산 판정	세트 제목 · 공통 비밀 · 공개 힌트
12 일 일정과 모든 ID	세트 사건 제목 · 요약 · 신문 리드
—	당일 시장 헤드라인(<code>marketHeadline</code>)

생성 범위는 위 오른쪽 열이 전부입니다. 의뢰 문구, 경매 진행 문구, 결산 요약, 엔딩 표현, 경쟁자 말투는 생성하지 않고 게임에 고정 작성되어 있습니다. 문서가 실제보다 넓게 읽히지 않도록 필드 이름을 함께 적었습니다.

이 경계 덕분에 모델이나 프롬프트가 바뀌어도 경제 규칙과 저장 데이터는 그대로 유지됩니다. 생성 결과에 `basePrice`, `trueValue`, `reward`, `multiplier`, `probability` 같은 금지 필드가 포함되면 검증 단계에서 실패 처리됩니다.

3 2 단계 생성 구조

3.1 런 시작 생성

게임 시작 시 한 번, 프레임 1 회 + 세트 12 회로 나누어 호출합니다.

호출	만드는 것
프레임	런 전제(<code>premise</code>), 12 일 시장 흐름(<code>marketArc</code> — 날짜별 헤드라인과 분위기)

호출	만드는 것
세트 × 12	세트 제목, 공통 비밀, 공개 힌트, 과거 사건 제목 · 요약 · 신문 리드

세트 사건은 그 세트에 속한 물품 이름 둘 이상을 명시적으로 엮어 만듭니다. 이것이 플레이어가 세트를 추론하는 재료입니다.

나누어 호출하는 이유는 실패를 가두기 위해서입니다. 한 번에 만들면 한 세트가 규칙을 어겨도 12 개가 통째로 버려집니다. 나누어 호출하면 해당 세트만 다시 만들고, 2 회 재시도 후에도 실패하면 그 세트만 폴백으로 채운 뒤 나머지 11 개를 살릴 수 있습니다.

세트를 따로 만들면 사건이 서로 비슷해지므로, 이미 만든 사건 제목을 다음 호출의 프롬프트에 함께 실어 중복을 피했습니다. 실측에서 12 개 세트의 사건 제목이 12/12 모두 달랐습니다.

모델은 엔진이 전달한 세트 ID 를 추가하거나 변경할 수 없습니다.

3.2 일일 생성

하루치를 한 번에 만들고, 실패한 항목만 다시 만듭니다.

호출	만드는 것
하루치 1 회	당일 시장 헤드라인(<code>marketHeadline</code>), 정확히 8 개 LOT 의 <code>displayName</code> · <code>description</code> · <code>rumor</code> · <code>setHint</code> · <code>npcReaction</code>
복구 (필요할 때만)	검증에 걸린 LOT 하나만 다시 생성

현재 날짜가 진행될 때 **2 일** 뒤까지 미리 준비해 플레이 중 대기를 줄였습니다. 저장 슬롯을 고르는 동안 런 `blueprint` 가 완성되므로, 실제 플레이에서는 생성을 기다리는 시간이 거의 드러나지 않습니다.

3.3 실패 처리

엔진 입력 생성

- 모델 호출
- JSON · ID · 필수 필드 · 금지 수치 · 문구 품질 검증
- 실패한 항목만 선별해 재생성 (하루치 전체가 아니라 해당 LOT 하나)
- 그래도 실패하면 다음 공급자로 전환
- 모두 실패하면 사전 준비된 폴백 문구 사용
- 검증된 결과만 게임에 공급

실패를 그 자리에 가둡니다. 이전에는 한 LOT 이 규칙을 어기면 하루치가 통째로 버려졌습니다. 지금은 `dailyRepairIndices` 가 고칠 자리만 선별해 그 LOT 만 다시 만들고, 나머지 결과는 그대로 사용합니다. 런 `blueprint` 도 같은 방식으로 프레임과 12 개 세트를 나누어 만들어, 한 세트가 실패해도 나머지 11 개는 남습니다.

재시도해도 결과가 달라지지 않는 실패는 즉시 중단합니다. 401 · 403 · 429 는 초 단위로 다시 호출해도 같은 응답이 오므로, 해당 요청에서는 그 공급자를 포기하고 다음 공급자로 넘어갑니다.

3.4 라우터 — 게임은 엔드포인트 하나만 압니다

게임은 어떤 모델을 사용하는지 알지 못합니다. 공급자 선택 · 순서 · 타임아웃 배분 · 복구 · 폴백은 전부 AWS Lambda 라우터가 처리하고, 게임은 엔드포인트 하나에 JSON 을 전달합니다.

이렇게 구성한 이유는 보안입니다. 배포본은 브라우저에서 동작하는 HTML 이라 API 키를 포함할 수 없습니다. 실제로 빌드 단계에서 `assertPublicGenerationConfig` 가 설정 파일에 `token` · `authorization` 같은 키가 있으면 빌드를 실패시킵니다. 키는 Lambda 환경변수에만 두고, 배포본에는 엔드포인트 주소만 포함됩니다.

■ 공급자 순서는 모드마다 다릅니다

모드	1 순위	2 순위
일자 생성	<code>gpt-5.6-luna</code>	<code>gpt-4o-mini</code>
런 blueprint	<code>gpt-4o-mini</code>	<code>gpt-5.6-luna</code>

측정 결과에 따라 정한 순서입니다. 일자 생성에서 `gpt-4o-mini` 는 두 번 연속 계약을 위반했고(`unsafe ending · does not match category`), 항목 복구를 거쳐도 통과하지 못했습니다. 반면 blueprint 에서는 `gpt-4o-mini` 가 네 번 모두 성공했습니다. 실패 가능성이 확인된 공급자를 매번 먼저 호출할 이유가 없다고 판단했습니다.

blueprint 순서는 변경하지 않았습니다. 근거가 없는 부분까지 함께 바꾸면, 이후 문제가 생겼을 때 원인이 둘로 늘어나기 때문입니다.

■ 타임아웃은 게이트웨이 예산 안에서 배분합니다

API Gateway 통합 타임아웃은 30 초입니다. 두 공급자를 순차로 호출하고도 그 안에 끝나야 합니다.

일자 생성 : luna 20 초 + mini 8 초 = 28 초
런 세트 : 조각이 작아 7~9 초씩, 대신 13 회 호출

일자 생성은 성공 지연 중앙값이 10.3 초인데도 상한을 20 초로 둔 것은 꼬리 지연 때문입니다. 상한을 더 올리면 2 순위를 호출할 시간이 없어지고, 낮추면 느린 판이 그대로 실패합니다. blueprint 는 13 회 호출하므로 상한을 키우면 총 시간이 게이트웨이 예산을 넘습니다.

■ 재시도가 무의미한 실패는 즉시 중단합니다

401 · 403 · 429 는 초 단위로 다시 호출해도 같은 응답이 옵니다. 그런데 초기 구현은 LOT 마다 재시도를 붙여 계속 호출했고, 키가 잘못되었을 때 일자 1 건에 실패 호출이 17 건 발생했습니다. 지금은 이 셋 중 하나가 나오면 해당 요청에서 그 공급자를 중단하고 다음으로 넘어가며, 17 건에서 최대 5 건으로 줄었습니다.

중단하는 쪽을 선택한 이유가 하나 더 있습니다. 계속 호출해서 얻는 결과물은 대부분이 폴백 문구인데 헤더에는 해당 공급자 이름이 찍힌 응답이었습니다. 실측에서 한 공급자가 정확히 그런 응답을 반환했습니다(8 개 중 2 개만 생성). 그럴 바에는 다음 공급자에게 기회를 주는 편이 낫다고 판단했습니다.

■ 제외한 공급자도 기록으로 남깁니다

무료 등급 공급자 하나를 1 순위로 두었다가 제외했습니다. 조직 단위 분당 토큰 한도에 걸려 8 개 중 6 개가 폴백으로 채워졌고, 모델을 더 작은 것으로 바꾸어도 같은 한도에 걸렸습니다. 모델 교체로 우회되는 문제가 아니라, 이 구조의 토큰 유입을 감당하지 못하는 문제였습니다.

코드는 삭제하지 않고 환경변수 한 줄로 되살릴 수 있게 남겨 두었습니다. 설계 결정이 아니라 용량 제약이므로, 조건이 바뀌면 되돌릴 수 있어야 한다고 보았습니다.

4 VSL, 게임 엔진, AI, Team Loop 의 역할

VSL

씬 목록, 화면 플로우, 좌표와 크기, UI 요소, 사운드와 전환의 골격을 만듭니다. 최종 VSL 결과는 어댑터와 매핑 파일을 통해 게임 상태에 연결합니다.

게임 엔진

12 일 일정, 96 개 LOT, 입찰, 판매, 의뢰, 대출, 파산, 결산과 최종 유물 경매를 결정론적으로 처리합니다. 같은 시드면 같은 일정과 같은 판정이 나옵니다.

생성 AI

엔진이 확정한 구조와 ID 를 바꾸지 않고, 플레이어에게 보이는 표현 콘텐츠를 채웁니다.

Team Loop 와 검증 하네스

생성 결과가 스키마, ID, 개수와 역할 경계를 지키는지 검사하고, 게임 회귀 테스트까지 함께 실행합니다. 도구 자체가 게임의 주인공은 아니며, AI 생성물을 실제 게임에 안전하게 연결하기 위한 제작 안전장치입니다.

사람

위 넷은 모두 도구입니다. 사람은 무엇을 만들지 정하고, 나온 결과를 보고 다음을 정합니다.

이 프로젝트에서 사람과 AI의 분업은 다음과 같이 나뉘었습니다.

사람이 한 일	AI가 한 일
무엇이 필요한지 묻고 정합니다	코드를 작성합니다
나온 결과를 보고 채택 여부를 정합니다	측정 도구를 만들고 실행합니다
어긋난 곳을 지적하고 방향을 조정합니다	문서와 인수인계 자료를 작성합니다
무엇을 포기할지 정합니다	대안과 근거를 제시합니다

이 방식이 성립하려면 검증이 자동이어야 합니다. 사람이 코드를 한 줄씩 읽어 확인하지는 않기 때문입니다. 그래서 판단이 필요한 자리마다 기계가 읽을 수 있는 근거를 남기도록 만들었습니다. 생성 결과는 검증기가, 게임 규칙은 테스트 109 건이, 연결 지점은 `npm run audit` 이, 성능과 품질은 배포본에 직접 요청을 보내는 측정 도구가 확인합니다.

측정이 사람의 판단을 뒤집은 적도 있습니다. 스키마를 줄인 뒤 8 판을 측정하고 “자연이 개선되었다”고 판단했는데, 8 판을 더 측정하니 결과가 뒤집혔습니다(7.6). 코드를 읽어서가 아니라 표본이 늘어나면서 잘못된 판단이 드러난 것입니다. 이 문서의 수치를 신뢰할 수 있는 근거도 여기에 있습니다.

한계도 분명합니다. 사람이 구현 세부를 직접 읽지 않으므로, 검증이 확인하지 않는 영역은 늦게 발견됩니다. 실제로 로딩 화면이 3 픽셀로 찌그러져 보이지 않던 결함이 있었는데, 테스트도 검증기도 이를 잡지 못했고 브라우저로 직접 열어본 뒤에야 발견했습니다. 자동 검증은 계약을 지키지만, “보기에 맞는가”는 확인하지 못합니다.

5 사용 도구와 활용 내역

5.1 게임 안에서 동작하는 것 (플레이할 때마다 실행)

도구 / 모델	버전	무엇을 하는가
OpenAI gpt-5.6-luna	모델명이 곧 버전	일자 생성 1 순위입니다. 하루 8 LOT의 설명 · 소문 · 세트 힌트 · NPC 반응을 만듭니다
OpenAI gpt-4o-mini	모델명이 곧 버전	런 blueprint 1 순위이자 일자 생성 2 순위입니다. 12일 시장 흐름과 세트 사건을 만듭니다

도구 / 모델	버전	무엇을 하는가
AWS Lambda + API Gateway	Node.js 22 런타임 · 512MB	위 두 모델을 감싸는 라우터입니다. 공급자 순서 · 타임아웃 배분 · 항목 단위 복구 · 폴백을 여기서 처리하며, 게임은 이 엔드포인트 하나만 압니다

게임 자체에는 런타임 의존성이 없습니다. 브라우저 표준 API 만 사용하며, 위 호출도 `fetch` 하나입니다.

5.2 콘텐츠 제작에 사용한 것 (개발 중 실행, 결과물만 게임에 포함)

도구	버전	무엇을 만들었나
OpenAI 이미지 생성	GPT-5.6 Sol (모델명이 곧 버전)	경매품 스프라이트 240 개(60 품목 × 4 등급), 씬 배경 11 종
OpenMusic AI	웹 서비스	타이틀곡 <code>Blue Harbor Ledger</code>
View Spec Lite (VSL)	v6.4 (Runtime/reference/vsl-v6.4/)	씬 목록 · 화면 플로우 · 좌표와 크기 · UI 요소 · 사운드와 전환의 골격입니다. 결과는 <code>contracts/vsl-map.template.json</code> 을 통해 런타임 DOM 에 연결됩니다. 이 편집 툴 자체도 팀이 AI 와 함께 제작했습니다. 씬을 등급 체계로 계층 관리하고, 연결선이 겹치지 않도록 색과 형태를 나누고, 편집 결과를 JSON 으로 내보내는 기능을 필요할 때마다 요구하고 수정하며 만들었습니다

5.3 개발과 검증에 사용한 것 (게임에는 포함되지 않음)

도구	버전	무엇을 했나
Codex	—	모듈형 웹 런타임, 데이터 계약, 테스트, 측정 도구를 작성했습니다
Claude Code	Opus 5	AWS 라우터 이전, 계약서 절 분리, 스키마 감축, 폴백 문구 은행 구축, 실측과 문서 갱신을 담당했습니다
Team Loop	스키마 v1 (2026-08-02 초기화)	생성 하네스 실행과 스키마 · 금지 필드 · 경로 범위 · 회귀 검증을 수행했습니다

도구	버전	무엇을 했나
밸런스 실험 엔진	Team Loop 내장	이 게임의 밸런싱랩을 범용 톨로 개량한 것입니다. 후보 파라미터를 시드별로 시뮬레이션해 단일 최선안과 Pareto 후보를 남깁니다. 워커 스레드로 동작하고 작업이 중단되어도 이어서 재개하며, 실험마다 결정 근거 · 패치 · 지표 · 차트를 자동으로 묶어 보관합니다. 수치를 채택할 때는 평균 하나가 아니라 시드별 분포 · 실패율 · 정책 비교 · 목표 위반 을 함께 통과해야 합니다
qwen3:14b / Ollama	qwen3:14b	로컬에서 2 단계 생성 구조와 계약 준수를 실험했습니다. 현재 게임은 이 경로를 사용하지 않으며 , 구조 검증용이었습니다
Node.js	24.14.1	로컬 생성 서버, 테스트, 측정 도구를 실행했습니다
esbuild	0.25.12	Lambda 번들 빌드 전용이며, 게임 실행에는 포함되지 않습니다

세 층을 나눈 이유가 있습니다. 심사 관점에서 “AI 로 무엇을 만들었나”와 “AI 가 게임 안에서 무엇을 하나”는 서로 다른 질문이기 때문입니다. 5.1 은 플레이어가 매번 마주치는 것, 5.2 는 한 번 만들어 넣은 것, 5.3 은 제작 과정에서만 사용한 것입니다.

6 주요 프롬프트 및 계약 방식

장문의 자연어 지시를 매번 전달하는 대신, 고정 규칙을 하나의 계약서 파일로 모았습니다. 파일은 `Runtime/contracts/compact-generation-contract.txt` 하나뿐이며, 검증기(`generation-server.js`)가 같은 파일을 근거로 결과를 검사합니다. 지시와 검사가 갈라지지 않도록 계약을 두 곳에 두지 않는 것이 원칙입니다.

6.1 계약서를 절로 나누어 필요한 것만 전달합니다

계약서를 모드별 절로 나누어 필요한 부분만 전달하자 한 판 입력이 **30.4%** 줄었습니다. 파일은 하나로 유지해 지시와 검사가 갈라질 여지를 두지 않았습니다.

계약서에는 `@ALL` · `@RUN` · `@DAY` 표식이 있고, 호출 모드에 따라 공통 절과 해당 모드 절만 조립해 전달합니다.

<code>@ALL</code>	227 자	JSON·한글 규칙, ID 보존, 시대 배경	
<code>@RUN</code>	887 자	런 스키마, 세트 사건 규칙	→ 런 호출에는 1,114 자
<code>@DAY</code>	1,116 자	일자 스키마, 길이·어미·카테고리 어휘	→ 일자 호출에는 1,343 자

이 작업이 필요했던 이유는 다음과 같습니다. 생성을 항목 단위로 쪼개자 호출 수가 늘었고, 쪼개기 전에는 한 번만 실리던 계약서가 한 판에 121 번 실리게 되었습니다. 그때 계약서가 전체 입력의 74%를 차지했고, 그중 상당량은 해당 호출에 쓰이지 않는 규칙이었습니다. LOT 하나를 만드는 호출이 세트 사건 규칙 695 자를 매번 함께 전달하고 있었습니다.

절을 나눈 뒤 한 판 입력이 **30.4%** 줄었습니다. 파일은 여전히 하나여서 갈라질 여지가 없고, 표식이 없는 계약서를 만나면 전부 @ALL 로 편집되어 이전과 동일하게 동작합니다.

표식 오타는 규칙을 조용히 사라지게 만듭니다. 어느 절에도 실리지 않은 줄이 생기면 모델은 규칙이 사라진 줄 모르고 응답하고, 검증기만 뒤늦게 걸러냅니다. 그래서 모든 줄이 적어도 한 모드에 실리는지를 테스트로 확인합니다.

6.2 실제 계약서 (일자 생성 부분)

요약이 아니라 모델에게 그대로 전달되는 문장입니다.

```
Return one JSON object only. Write every user-facing text value in Korean.
Never output numeric game rules. Copy IDs exactly and preserve input order.

DAY: {schemaVersion:"1.0",day,marketHeadline,
      lots:[exactly 8 {lotId,displayName,description,rumor,setHint,npcReaction}]}.
```

Daily limits per item: displayName 20 Korean characters, description 70, rumor 45, setHint 25, npcReaction 45. Use one sentence per text field. Copy each input baseName exactly into displayName. Describe only visible material, engraving, discoloration, repair, provenance mark, or wear. Description must be one factual observation ending with one of: "남아 있다.", "보인다.", "확인된다.", "이어진다.", "드러난다." Vary the observed feature across all eight items and do not reuse the same phrase three times. Never claim magic, supernatural powers, translation, navigation, appraisal, market value, or gameplay effects as fact. Category vocabulary: CER uses 유약/굽/몸체/손잡이/뚜껑/항아리/도자; CLK uses 문자판/바늘/태엽/케이스/시계/크로노미터/유리돔/골격; ... Describe the artifact, not the person or place depicted by its name.

6.3 실제 생성 결과

계약이 지키는 것은 형식이고, 값을 채우는 것은 내용입니다. 아래는 배포된 엔드포인트가 실제로 만든 세트 하나이며, 손대지 않은 원문 그대로입니다.

필드	생성 결과
세트 제목	공회와 은밀한 유물
공통 비밀	두 개의 보석이 한 번의 사건을 묘사한다.
공개 힌트	티아라와 반지가 같은 날 만들어졌다.
사건 제목	티아라와 반지의 은밀한 연결

필드	생성 결과
사건 요약	티아라와 반지는 한 명의 장금사가 제작한 것으로 확인되었다. 그는 두 개의 보석을 서로 다른 주문자에게 판매했지만, 그 둘은 이후 한 번의 사건에서 연결되었다.
신문 리드	티아라와 반지의 연관성, 한 명의 장금사가 그 비밀을 알고 있다.

여기서 계약이 강제된 것은 세 가지입니다. 사건이 세트에 속한 물품 이름 둘 이상을 명시적으로 엮을 것, 마법 · 유령 · 예언 같은 초자연 요소를 쓰지 않을 것, 가격 · 보상 · 확률을 쓰지 않을 것입니다. 실제로 “장금사가 두 주문자에게 팔았다”는 평범한 인간사이며 수치가 하나도 포함되지 않았습니다.

신문 리드가 요약을 그대로 옮기지 않는 것도 계약입니다. 같은 사건을 다른 각도에서 쓰게 해, 플레이어가 두 번 읽을 이유를 만듭니다.

LOT 쪽 결과는 7.4에 런 두 개를 나란히 정리했습니다.

6.4 형식은 지시가 아니라 검증기가 잡습니다

문장으로 “마침표는 하나” 같은 형식을 지시해 보았지만, 이전과 같은 비율로 위반이 발생했습니다(8 건 중 1 건). 그래서 형식 강제는 프롬프트에서 제외하고 검증기에 맡겼습니다.

검증기가 확인하는 것은 스키마 · ID · 개수 · 금지 수치에 더해 문구 품질입니다. 길이 초과, 허용 밖 어미, 카테고리 어휘 불일치, 한 문장 위반, 여덟 항목 간 중복이 모두 자동으로 검출됩니다. 검출된 항목은 6.5의 복구 호출로 넘어갑니다.

6.5 복구는 실패한 항목에만 전달합니다

전체를 다시 만들지 않고 해당 LOT 하나만 다시 만듭니다. 복구 프롬프트에는 위반한 규칙과 그 LOT 입력만 씁니다.

```
RETRY_ERRORS:
  lot 5 description has unsafe ending
  <해당 모드의 계약 절>
INPUT LOT: {lotId, baseName, category, grade, setId}
```

실측에서 8 개 중 6 개가 이 경로로 복구되었습니다. 하루치를 통째로 버리던 때와 비교하면, 같은 실패에서 살릴 수 있는 양이 크게 다릅니다.

7 측정 결과

모든 수치는 배포된 엔드포인트에 실제로 요청을 보내 얻은 값입니다. 시뮬레이션이나 추정이 아닙니다. 측정일은 2026-08-07~08입니다.

7.1 배포 API 성공률과 지연

24 판 중 23 판이 실제 생성으로 반환되었고, 계약 검증 실패는 0 건입니다. 남은 1 판도 게임이 멈추지 않고 폴백 문구로 이어졌습니다.

일자 생성(하루 8 LOT)을 시드만 바꾸어 순차로 24 판 호출한 결과입니다.

지표	결과
실제 생성	23 / 24 판
폴백으로 내려간 판	1 / 24
계약 검증 실패	0 / 24 판
지연 중앙값	10.3 초
지연 범위	8.0 ~ 20.1 초

런 blueprint(프레임 + 세트 12 개)는 15.1 초에 완료되었고, 12 개 세트의 사건 제목이 **12/12** 모두 서로 달랐습니다. 세트를 나누어 만들 때 이미 사용한 제목을 프롬프트에 실어 중복을 피하는 장치가 정상 동작한 결과입니다.

폴백으로 내려간 1 판의 사유도 로그로 확인했습니다. 공급자 두 곳이 모두 응답 시간을 넘긴 경우였고(20 초 · 8 초), 인증 오류나 계약 위반은 아니었습니다.

7.2 입력량 감축

같은 결과를 얻는 데 드는 입력을 두 차례 줄여, 한 판 기준으로 계약서에서 **30.4%** · 스키마에서 **15%**를 절감했습니다. 줄인 뒤 다시 측정해 계약 준수가 나빠지지 않은 것도 확인했습니다.

조치	효과
계약서를 모드별 절로 분리	한 판 입력 30.4% 감소
스키마의 항목 정의 반복 제거	일자 스키마 3,997 → 798 자 (80% 감소)
"	런 스키마 6,248 → 1,126 자 (82% 감소)

두 번째 조치는 스키마가 8 개 LOT의 정의를 여덟 번 반복하던 것을 하나로 합친 것입니다. 일자 호출 하나가 6,117 → 2,918 자가 되었고, 한 판 전체 입력이 약 **15%** 더 줄었습니다. 줄인 뒤 16 판을 다시 측정해 계약 준수가 나빠지지 않았음을 확인했습니다.

7.3 폴백 품질

폴백 문구의 중복이 구조적으로 사라졌습니다. **600** 일치 검사에서 하루 안 중복 **0** 건, 계약 검증 실패 **0** 건입니다. 이전에는 매일 최소 두 쌍이 글자 그대로 같았습니다.

AI 가 실패해도 게임이 멈추지 않는다는 점은 이전부터 확인된 사실입니다. 이번에 새로 측정한 것은 폴백으로 나오는 문구 자체의 품질입니다.

지표	이전 (문자열 조합)	현재 (사전 준비 문구)
하루 8 LOT 중 중복 설명	매일 최소 2 쌍	0 건
하루 평균 서로 다른 문장 어미	2 가지	3.99 가지
하루 평균 서로 다른 첫머리	—	7.26 / 8
계약 검증 실패	0	0 / 600 일

50 개 시드 × 12 일 = **600** 일치를 검사한 결과입니다. 실제 생성의 어미가 8 판 평균 4.6 가지이므로, 폴백이 실제 생성에 근접한 다양성을 확보했다고 볼 수 있습니다.

■ 폴백 문구를 만든 방법

새로 지어내지 않고, 지난 실제 생성물을 재활용했습니다. 이미 비용을 치렀고 계약을 통과한 문장들이기 때문입니다. 거르는 단계는 네 가지입니다.

1. 실제 검증기로 검사합니다. 후보를 여덟씩 묶어 **qualityErrors** 에 그대로 입력하고, 검출된 항목만 제거합니다. 카테고리 어휘 · 어미 · 한 문장 규칙이 모두 그 함수에 있으므로, 여기서 다시 작성하면 규칙이 갈라집니다
 2. 어미만 다른 중복은 어간 기준으로 하나만 남깁니다
 3. 카탈로그 60 종 품목 이름이 포함된 문장은 제외합니다
 4. 품목 머리 명사가 포함된 문장은 예비로 강등합니다. “소금단지의 뚜껑에”가 청자 항아리에 붙는 것을 막기 위해서입니다
- 3 · 4** 단계는 실제로 발생한 사고에서 나왔습니다. 배포 후 확인 과정에서 청자 항아리에 “제비문이 찻잔의 손잡이에 새겨져 있고”가 붙었습니다. 찻잔은 카탈로그에 없는, 모델이 지어낸 사물이라 이름 목록으로는 걸러지지 않았습니다. 조사가 붙은 형태로만 거르는 목록을 따로 두어 해결했습니다.

카테고리마다 문장이 부족하면 해당 카테고리만 **8** 개인 하루를 만들어 배포본에 요청해 채웠습니다. 회화가 6 개뿐이던 것을 65 개로 늘리는 방식입니다.

남은 한계도 함께 밝힙니다. 카테고리 단위로 재사용하므로, 황동 물건에 은제 설명이 붙는 재질 어긋남은 남아 있습니다. 품목별로 문구 은행을 나누면 없앨 수 있지만 그러면 문장 수가 부족해집니다. 현재 방식은 **100%** 티가 나던 이전 템플릿보다 개선되었으나, 완전하지는 않습니다.

7.4 런마다 달라지는가 — 같은 품목, 다른 런

같은 품목이 런마다 다른 설명 · 소문 · 세트 힌트를 받습니다. 1 장에서 세운 첫 목표가 실제로 지켜지는지, 시드만 바꾸어 1 일차를 두 번 생성해 확인한 결과입니다.

엔진의 몫 — 일정 구성이 달라집니다. 8 개 LOT 중 겹친 품목은 2 개뿐이었습니다.

생성의 몫 — 겹친 품목조차 다른 문장을 받습니다. 이 부분이 핵심입니다.

붉은 장갑의 배우 (두 런에 모두 등장)

	런 A	런 B
설명	액자 안쪽에 벗겨진 안료와 얇은 덧칠 자국이 남아 있다.	화폭 가장자리에 붉은 안료가 번졌고 액자에 작은 보수 흔적이 남아 있다.
소문	무대 뒤에서 급히 손질했다는 말이 있다.	초연 직후 극장 관리인이 말렸다는 소문이 있다.
세트 힌트	붉은 색조 그림의 연결점	무대 초상화 연작

향로문 금은 상감함

	런 A	런 B
설명	황동 표면의 은제 상감선 사이로 어두운 녹이 확인된다.	금속 표면에 금은 상감선과 옅은 녹이 남아 있다.
소문	문양의 일부가 뒤늦게 덧대어졌다는 소문이다.	경매장 장부에 봉인 표식이 있었다는 소문이다.
세트 힌트	금속 공예품 사이의 빈 고리	금속 공예품과 어울림

세트 힌트가 달라지는 점이 게임에서 가장 중요합니다. 플레이어는 이 문구로 어떤 물품끼리 묶이는지 추론하므로, 같은 물품이라도 런마다 다른 실마리를 받게 됩니다.

덜 달라지는 것도 있습니다. 시장 헤드라인은 두 런이 거의 같았습니다. “안개 속 경매장에 오래된 항구의 흔적이 모였다”와 “안개 낀 경매장에 항구의 오래된 흔적이 모인다.”가 그 예입니다. 하루에 하나뿐인 필드라 표현 폭이 좁고, LOT 문구만큼 갈리지는 않습니다.

7.5 게임 및 하네스 검증

■ 게임 런타임 테스트 109 건 전부 통과

- 생성 연결 점검(`npm run audit:generation`) 오류 0 건
- 900 개 시드에서 86,400 LOT 일정 생성, 잘못된 12 일 일정 0 건
- 생성 결과의 엔진 소유 수치 침범 0 건
- 브라우저에서 배포본을 열어 런 blueprint 와 1~3 일차 생성 수신까지 확인

7.6 수치 해석의 한계

표본이 작습니다. 24 판은 통계라기보다 관찰에 가깝습니다. 실제로 이 프로젝트는 같은 함정을 한 차례 겪었습니다. 스키마를 줄인 뒤 8 판을 측정하고 “지연 중앙값이 10.9 초에서 9.4 초로 개선되었다”고 기록했는데, 8 판을 더 측정하니 11.4 초가 나와 합산 중앙값이 10.3 초였습니다. 8 판으로 성능 변화를 단정할 수 없다는 것이 이 과정에서 확인한 사실이며, 위 표의 지연 수치도 같은 기준으로 읽어야 합니다.

그래서 주장하지 않는 것과 주장하는 것을 구분했습니다.

- 주장하지 않습니다 — 지연이 개선되었다, 폴백률이 낮아졌다
- 주장합니다 — 입력량이 줄었고(측정 가능), 계약 준수가 24 판 내내 유지되었으며(0/24), 폴백 문구의 중복이 구조적으로 사라졌습니다(600 일 검사)

8 실제 프로젝트에 미친 영향

적용 전

- UI 목업, 기획 문서와 밸런싱 자료가 실제 게임 상태와 분리되어 있었습니다.
- 생성 AI 요청 · 응답의 명시적 계약과 실패 처리 방식이 없었습니다.
- VSL 최종 결과를 받기 전에는 시스템 연결 작업을 시작하기 어려웠습니다.

현재

- VSL 과 무관하게 12 일 진행을 검증할 수 있는 모듈형 런타임이 존재합니다.
- 경매 · 판매 · 의뢰 · 경제 · 대출 · 결산이 같은 상태 데이터를 공유합니다.
- VSL 요소를 교체할 어댑터와 좌표 매핑 계약이 준비되어 있습니다.
- 생성 · 검증 · 재시도 경로를 실제 모델로 확인했습니다.
- AI 가 없어도 폴백으로 12 일 진행이 중단되지 않습니다.

무엇이 이 속도를 만들었나

이 프로젝트에서 실제로 효과가 컸던 것은 특정 모델이 아니라, 판단할 때마다 근거를 남기게 만든 구조였습니다.

- 계약을 한 곳에만 둡니다. 지시와 검사가 같은 파일을 참조하므로 갈라지지 않습니다. 실제로 검증기를 복제해 둔 도구 하나가 계약에 없는 필드를 계속 요구해 올바른 결과를 탈락시킨 적이 있었고, 그 이후로 복제를 금지했습니다
- 실패를 항목 단위로 가둡니다. 한 LOT의 오류가 하루치를 버리지 않습니다
- 결정을 문서에 남깁니다. 되돌린 이유, 잘못된 판단, 표본이 부족해 단정하지 못한 부분까지 기록합니다. 다음 사람이 같은 실험을 반복하지 않도록 하기 위해서입니다
- 눈으로 확인할 것은 눈으로 확인합니다. 자동 검증이 닿지 않는 영역이 있다는 것을 결함 하나로 배웠습니다(4 장)

사용하지 않기로 결정한 것도 있습니다. 팀은 이 게임과 별개로 오케스트레이션과 루프 엔지니어링 두 갈래의 제작 도구를 만들어 왔고, 둘은 **Team Loop** 하나로 통합되었습니다. 갈라져 있던 융합 브랜치가 현재는 본류입니다. 다만 그 전체를 이 게임에 적용하려면 마감 안에 끝나지 않는다고 판단해, 밸런스 실험 엔진 한 가지만 분리해 사용했습니다. 파이프라인과 워크플로 구상도 세웠지만 같은 이유로 이번 범위에서는 제외했습니다.

도구를 더 넣는 것이 항상 이득은 아니었습니다. 이번에 실제로 값을 한 것은 계약 · 검증 · 측정처럼 결과를 확인해 주는 장치였고, 그 밖의 자동화는 마감이라는 제약 앞에서 비용이 더 컸습니다.

남은 과제도 같은 자리에 있습니다. 폴백 문구는 카테고리 단위로 재사용하므로 드물게 물품과 어긋난 표현이 나올 수 있고, 표본이 작아 단정하지 못한 수치가 아직 몇 개 남아 있습니다. 문서에 그대로 표시해 두었습니다.

9 외부 에셋 · 오픈소스 · 라이선스

항목	출처	라이선스 / 사용 조건	AI 생성 여부	적용 위치
경매품 스프라이트 240 개 (60 품목 × 4 등급)	팀 제작 — OpenAI 이미지 생성(GPT-5.6 Sol)	OpenAI 이용약관은 생성 결과물(Output)에 대한 권리를 사용자에게 귀속시킵니다. 상업적 사용에 제한이 없습니다	예 — AI 생성	품목 카드 · 경매 화면
썸 배경	팀 제작 — OpenAI 이미지 생성(GPT-5.6 Sol)	위와 같습니다	예 — AI 생성	타이틀 · 도시 · 의뢰소 · 술집 · 거래소 · 상회 · 조합 · 전시관 · 경매 · 결산 · 결과 화면

항목	출처	라이선스 / 사용 조건	AI 생성 여부	적용 위치
UI 골격 · 목업	View Spec Lite (팀 도구)	팀 내부 도구입니다	아니오	씬 배치 · 좌표 · 플로우
BGM 6 곡 (Thatched Villagers, March of the Spoons, Midnight Tale, I Knew a Guy, Court of the Queen, Peaceful Desolation)	Kevin MacLeod (https://incompetech.com)	CC BY 4.0 — 아래 크레딧 문구가 필수입니다. 게임 내 루프 · 길이 · 음량 편집만 했고, 음원 자체를 별도 상품으로 재판매하지 않습니다	아니오	도시 · 의뢰소 · 거래소 · 상회 · 조합 · 술집 · 경매 · 유물 경매 · 결산
BGM Blue Harbor Ledger (타이틀)	팀 제작 — OpenMusic AI (https://www.openmusic.ai)로 생성	약관상 생성물의 권리는 생성한 사용자에게 귀속되며, 로열티 프리로 게시 · 배포 · 수익화 · 수정이 가능하고 출처 표기 의무는 없습니다. 서비스는 기반 모델 · 인프라에 대한 권리만 보유합니다	예 — AI 생성	타이틀 화면
SFX	Pixabay — AI 에이전트가 검색 · 선정하여 적용	Pixabay Content License — 상업 · 비상업적 사용 및 수정이 가능하고 출처 표기는 불필요합니다. 다만 원본 파일의 단독 재판매 · 재배포는 금지됩니다	아니오 — Pixabay의 기존 SFX 사용	게임 내 UI · 경매 · 알림 효과음 등
폰트	배포 없음 — 폰트 파일을 포함하지 않습니다	라이선스 대상이 아닙니다. CSS 가 이름만 지정하고(Batang/바탕, Noto Serif KR, Georgia, Inter, Noto Sans KR) 실제 렌더링은 사용자의 시스템 폰트를 사용합니다. @font-face · 웹폰트 로드가 없습니다	아니오	전체 화면
아이콘	UI 스프라이트에 포함	위 UI 항목과 같습니다	위와 같습니다	버튼 · 상태 표시

항목	출처	라이선스 / 사용 조건	AI 생성 여부	적용 위치
JavaScript 패키지	프로젝트 Runtime/package.json	런타임 의존성 0 개 — 게임은 브라우저 표준 API 와 Node.js 내장 모듈만 사용합니다. 빌드 도구로 esbuild(MIT) 하나를 개발 의존성에 두었습니다	아니오	게임 실행에는 포함되지 않음 (Lambda 번들 빌드 전용)

필수 크레딧 표기

Kevin MacLeod 의 6 곡은 CC BY 4.0 조건이므로 아래 문구를 그대로 표기합니다.

"Thatched Villagers", "March of the Spoons", "Midnight Tale", "I Knew a Guy",
"Court of the Queen", and "Peaceful Desolation" by Kevin MacLeod
(<https://incompetech.com>) are licensed under Creative Commons Attribution 4.0:
<https://creativecommons.org/licenses/by/4.0/>

Edited for in-game looping, duration, and level matching.

전체 소스 저장소는 https://github.com/sangjin615/NAN_2026_Team_Project 입니다. 문의 사항은 팀으로 연락 주시면
답변드리겠습니다.